URL SHORTENER WEB APPLICATION

Project Report

1. Introduction

In today's digital world, URLs often become long and complex, especially when they include tracking parameters, session IDs, or nested paths. Sharing such long URLs through messages, emails, or social media becomes inconvenient and error-prone. To solve this problem, URL Shortener applications are used to convert long URLs into short, easy-to-share links.

This project focuses on building a **basic URL Shortener Web Application** using **Flask (Python)** for the backend, **HTML, CSS, and Bootstrap** for the frontend, and a **database ORM** for storing URL history. The application allows users not only to shorten URLs but also to view previously shortened URLs.

2. Objectives of the Project

The main objectives of this project are:

1. To build a web application that converts long URLs into short URLs.
2. To store original URLs along with their shortened versions in a database.
3. To provide a user-friendly interface for shortening and copying URLs.
4. To maintain a history page showing all previously shortened URLs.
5. To validate whether the user-entered URL is correct before processing it.

3. Why a URL Shortener is Needed

Long URLs can be:

- Difficult to remember
- Inconvenient to copy and paste
- Unattractive when shared publicly

A URL shortener solves these problems by:

- Creating compact links

- Improving readability
- Making sharing easier with a single click
- Allowing storage and tracking of URLs

This project demonstrates how such a system works internally using basic web technologies.

4. Technologies Used

Frontend

- HTML – Page structure
- CSS – Styling
- Bootstrap – Responsive design and UI components

Backend

- Python
- Flask (Web framework for routing and request handling)

Database

- SQLite (or any relational DB)
- ORM (Flask-SQLAlchemy)

5. Project Architecture

The project follows a **simple MVC-like structure**:

- **Frontend (View)**
  Handles user interaction and displays results.
- **Backend (Controller)**
  Processes requests, validates URLs, generates short codes, and interacts with the database.
- **Database (Model)**
  Stores original URLs and their shortened versions.

6. Frontend Design

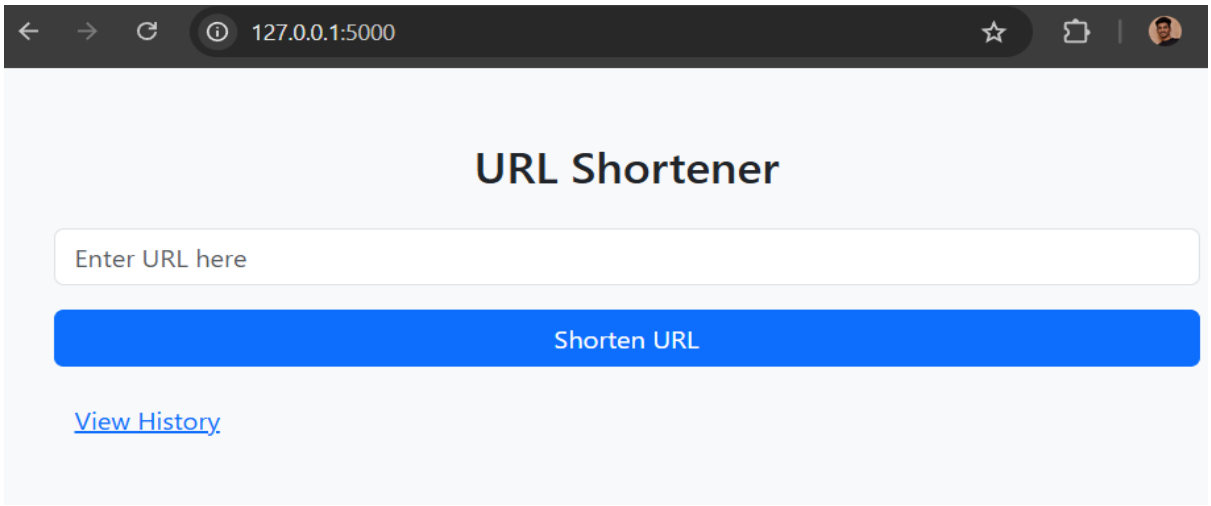
6.1 Home Page

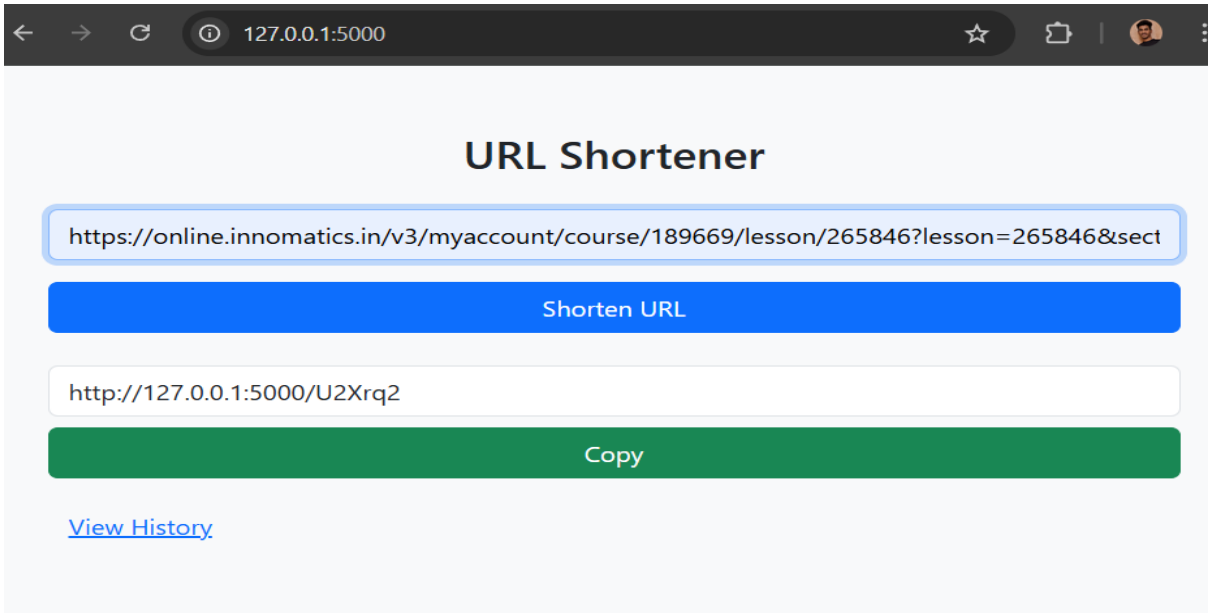
The Home Page contains:

- An input field where the user enters a long URL
- A **Shorten URL** button
- A text field displaying the shortened URL
- A **Copy** button to copy the shortened link to the clipboard

Workflow:

1. User enters a URL.
2. User clicks the **Shorten** button.
3. Shortened URL is displayed.
4. User copies the URL with one click.



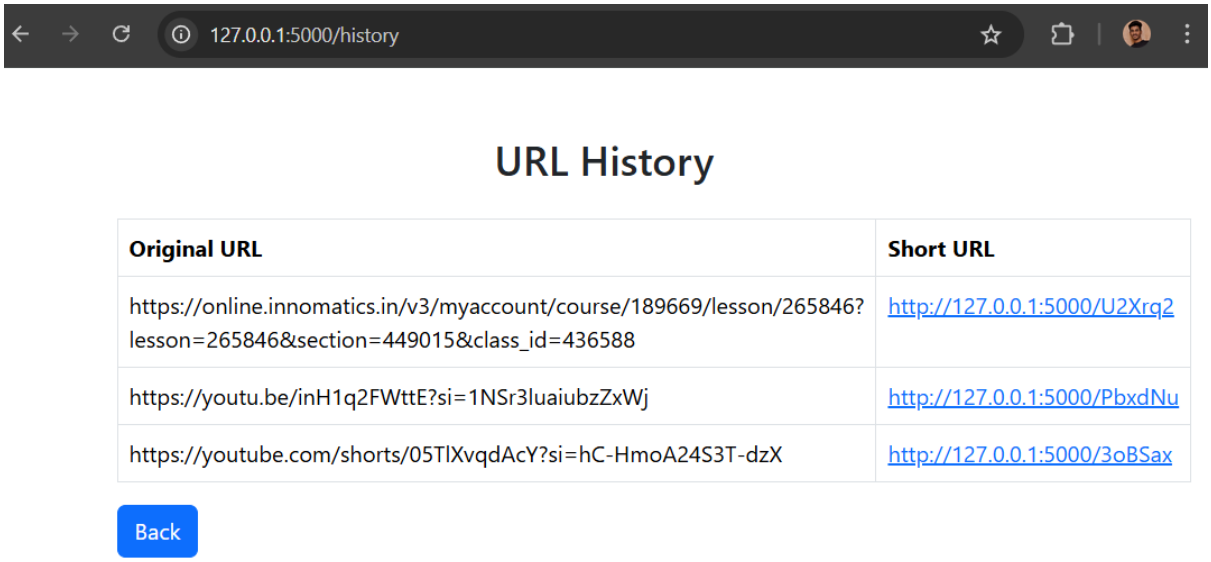


6.2 History Page

The History Page displays:

- A list of all original URLs
- Their corresponding shortened URLs

This data is fetched directly from the database and rendered in a table format using Bootstrap.



7. Backend Design and Workflow

7.1 URL Shortening Logic

To shorten a URL:

1. A unique ID is generated for each URL.
2. This ID is converted into a short alphanumeric string (Base62 encoding or random string).
3. The short string becomes part of the shortened URL.

Example:

Original URL: *https://www.example.com/some/very/long/path*

Short URL: *http://localhost:5000/abc123*

7.2 URL Redirection

When a user visits the shortened URL:

1. Flask captures the short code from the URL.
2. The database is queried for the original URL.
3. The user is redirected to the original URL using Flask's redirect mechanism.

8. Database Design

Column Name	Description
id	Primary key
original_url	Long URL entered by user
short_code	Generated short string
created_at	Timestamp

This table ensures persistent storage of URLs and enables the history feature.

9. URL Validation

To verify whether a URL entered by the user is valid:

Approach:

- Use Python's validators library
  or
- Use urllib.parse to check scheme (http, https) and domain

Example Validation Logic:

- Check if the URL starts with http:// or https://
- Ensure a valid domain name exists

Invalid URLs are rejected with an appropriate error message.

10. ORM Usage

Instead of writing raw SQL queries, an **ORM (Object Relational Mapper)** is used.

Advantages:

- Cleaner code
- Database-independent
- Secure against SQL injection
- Easy CRUD operations

Each URL entry is represented as a Python class mapped to a database table.

11. Project Workflow Summary

1. User enters a URL on the Home Page.
2. URL is validated.
3. A unique short code is generated.
4. Original URL and short code are stored in the database.
5. Shortened URL is displayed to the user.
6. User can copy the URL.

7. User can view all past URLs on the History Page.
8. Visiting a shortened URL redirects to the original URL.

12. Challenges Faced

- Generating unique short URLs
- Preventing duplicate entries
- Validating user input URLs
- Managing redirection efficiently
- Designing a clean UI with minimal complexity

13. Future Enhancements

- User authentication
- Click analytics
- Expiry time for short URLs
- Custom short URLs
- QR code generation
- API support

14. Conclusion

This project demonstrates a complete end-to-end web application using Flask and modern frontend tools. It successfully shortens URLs, stores them using an ORM, and provides a simple and intuitive interface for users. The project helped in understanding backend routing, database integration, URL validation, and frontend-backend interaction.